
cmakeと演習プログラムのビルド

Geant4 11.3 準拠

Geant4講習会資料: HEP/NP/Space 2025



KEK

大学共同利用機関法人

高エネルギー加速器研究機構

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません

目次

1. プログラム ビルド入門
2. Geant4でのcmakeの使い方
3. cmake使用のまとめ

はじめに

■ 本講義ではcmakeおよびmakeの詳細には立ち入らない

- cmake言語、make言語はそれ自体が複雑で習得には時間がかかる
- 本講習会で提供するCMakeLists.txtは汎用性を考慮して作られているので、ユーザが今後、独自のGeant4アプリケーションを作るときにも使える
 - ・ 最低のパラメータ設定は必要であるが、これに関しては後述
- まずはblackboxとして提供CMakeLists.txtを使い、必要に応じて内容を勉強すること勧める

■ makeおよびcmakeの標準参考文献をあげておく

- cmake: <http://www.cmake.org/documentation/>
- make: <http://www.gnu.org/software/make/manual/make.html>

プログラム ビルド入門

ビルド・ツールとは何か？

■ ビルドとはプログラムのソースコードから実行モジュールを作成すること

- 単純なプログラムではビルドは簡単

```
$ g++ helloWorld.cc  
$ a.out
```

- しかし、Geant4のアプリケーションのような膨大な数のソース・ファイルから成るプログラムではビルドは複雑で間違いが起きやすい

■ ビルド・ツールとは複雑なビルド手続きを自動化させるもの

- スクリプトに複雑な手続きを記述して、それを実行させることでビルドを実行

■ Linux環境でのビルドツール

- **make**が標準的なツールで**GNUmake**が最も一般的に使われる
- **Makefile**とよばれるファイルに**make言語**を用いてビルドの手続きを書く
- Geant4もversion9.4以前ではGNUmakeを直接使っていた

■ Linux環境以外でのビルドツール

- Windows: Visual Studioが用意するビルド環境
- Mac OSX: Xcodeが用意するビルド環境
- etc

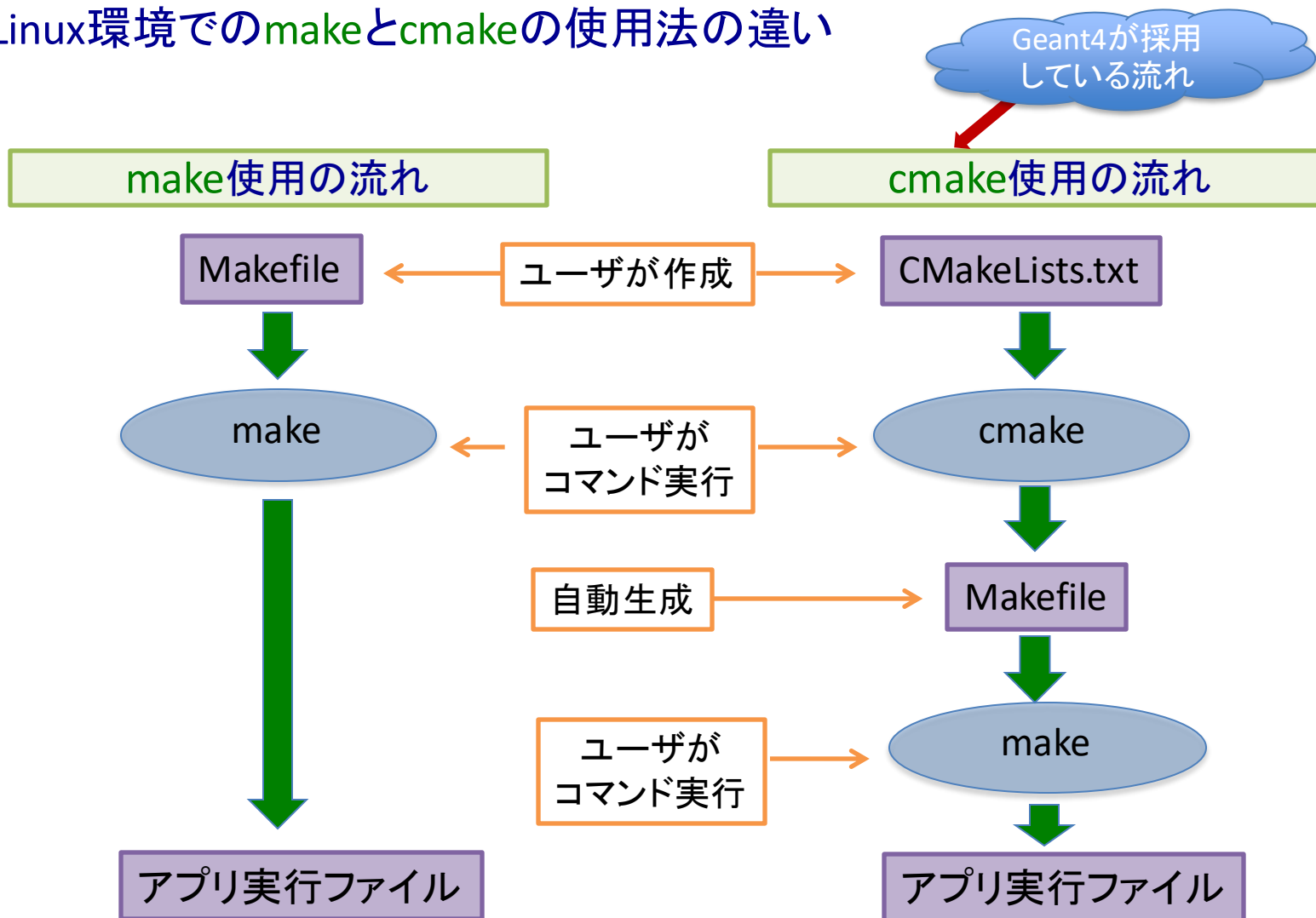
makeとcmake

- **make**はプログラムのビルドを自動化してくれるが、異なる計算環境 (linux/Unix, Windows, Mac OSX, etc)で統一的に使えるものではない
- **cmake**は異なる計算機環境でのビルドを一元的に管理するツール
 - cmakeを使えば、Geant4のビルド(ライブラリ、アプリケーションを含めて)を異なる計算機環境で統一的に行える
 - cmakeではCMakeLists.txtとよばれるファイルにcmakeコマンドを用いてビルドの手続きを書く
 - Geant4はversion 9.5からcmakeを採用
 - GNUmakeを使ったビルド手法はversion 10.0以降はサポートしていないので必ずcmakeを使うこと
- cmakeはlinux/unix環境ではMakefileを自動生成する
 - Windows環境ではmakeが使えないので、cmakeコマンドがVisual Studioのビルド・コマンドにマッピングされる

[注] WSL (Windows Subsystem for Linux)を使う場合はOK

makeとcmake - 続き

■ Linux環境でのmakeとcmakeの使用法の違い

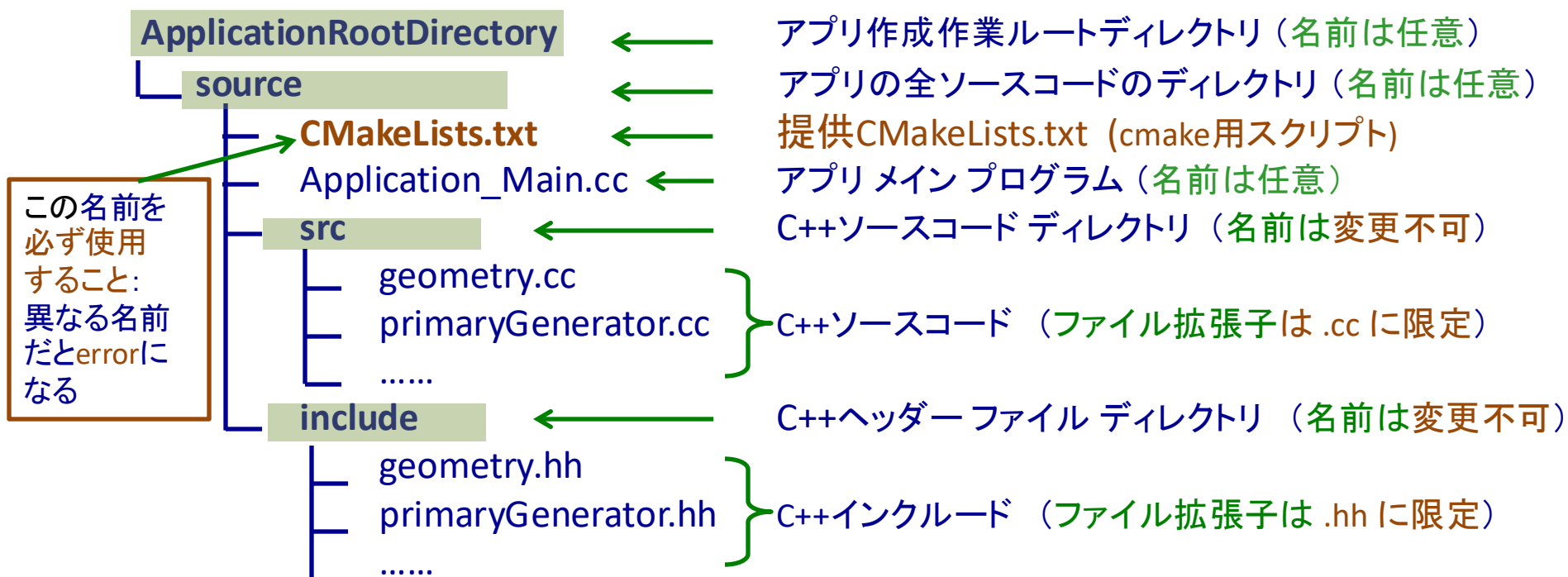


Geant4でのcmakeの使い方

演習で提供されるCMakeLists.txtを使うためのファイル環境

■ 演習で提供されるCMakeLists.txtは下図のファイル構造で使うことを前提としている

- アプリを作る場合、初心者ユーザはこの構造に従うことを推奨する
- 演習プログラムは全てこのファイル構造で構成されている
- cmake言語をある程度学べば、独自のファイル構造で作業ができるようになる



[注] 提供演習環境では拡張子が'.cc'および'.hh'でないファイルをcmakeは無視(ビルドの対象外)

ヘッダー・ファイルとソース・ファイル

- Geant4の様な大規模なプログラムでは、クラス定義がヘッダー・ファイルとソース・ファイルの二つに分離され記述される
 - **ヘッダー・ファイル**; クラスの定義が書かれたファイルで、メンバー関数は宣言のみで実装なし
 - 前スライドの**include フォルダ**ーにはユーザアプリで定義している全ヘッダーファイルが保存されている
 - 通常、ファイル名の拡張子として'.h' あるいは'.hh' がつく: [例] myClass.hh
 - **ソース・ファイル** : ヘッダーファイルで宣言されたメンバー関数の実装が書かれたファイル
 - 前スライドの**src フォルダ**ーにはユーザアプリで定義している全ソースファイルが保存されている
 - 通常、ファイル名の拡張子として'.c' あるいは'.cc' がつく: [例] myClass.cc
- クラス定義がヘッダー・ファイルとソース・ファイルに分離される理由は以下のとおり:
 - ライブラリ化されたモジュール(C++の標準ライブラリ、Geant4ツールキットなど)を使用する場合、ユーザは必要なヘッダーファイルを自分のプログラムに取り込む(includeする)だけで良い
 - ユーザはincludeさえすれば、クラスの定義の内容を知らなくても、クラスの公開メンバーを使用できる
- C++標準ライブラリの使用時のヘッダー・ファイルのinclude例:

```
1 #include <iostream>
2 #include <cmath>
3
4 int main() {
5     double value = std::sin(3.14159265/4.0);
6     std::cout << "value: " << value << std::endl;
7 }
```

標準ライブラリの入出力を使用するのに必要なヘッダーファイルの読み込み

標準ライブラリの数学関数を使用するのに必要なヘッダーファイルの読み込み

C++標準ライブラリの入出力およびsinと関数の使用例

提供CMakeLists.txtの使用方法

- 提供されているCMakeLists.txtを自分のアプリ作成に使用するときに変更しなければならないのは、以下に示すmainプログラム名のみ
 - 他の部分は一切触れる必要なし
 - cmakeを実行した際に使用するGeant4ライブラリは、ユーザの.bashrcで実行されているスクリプト 'geant4.sh' で設定済み

```
#####
# CMakeLists.txt: Geant4 Tutorial for Hep/Space Users
#####
cmake_minimum_required(VERSION 2.6 FATAL_ERROR)

#-----
# Setup Section: You have to set the following parameter
#-----
# Set the name of your main program
set( NAME_MAIN_PROGRAM Application_Main )

#-----
# Intact Section: You don't need to touch the following part
#-----
# Set the project name
project( ${NAME_MAIN_PROGRAM} )

# Find Geant4 package
.....
```

ここにcmakeしたいアプリのmainプログラム名を書く（プログラム名は任意）

- `set()`はcmake言語で変数の設定に使うコマンド
- `NAME_MAIN_PROGRAM`はcmake内部変数名（名前変更不可）
- mainプログラムのファイル名が `Application_Main.cc` なら `Application_Main` がmainプログラム名

[注] cmakeのシンタックス

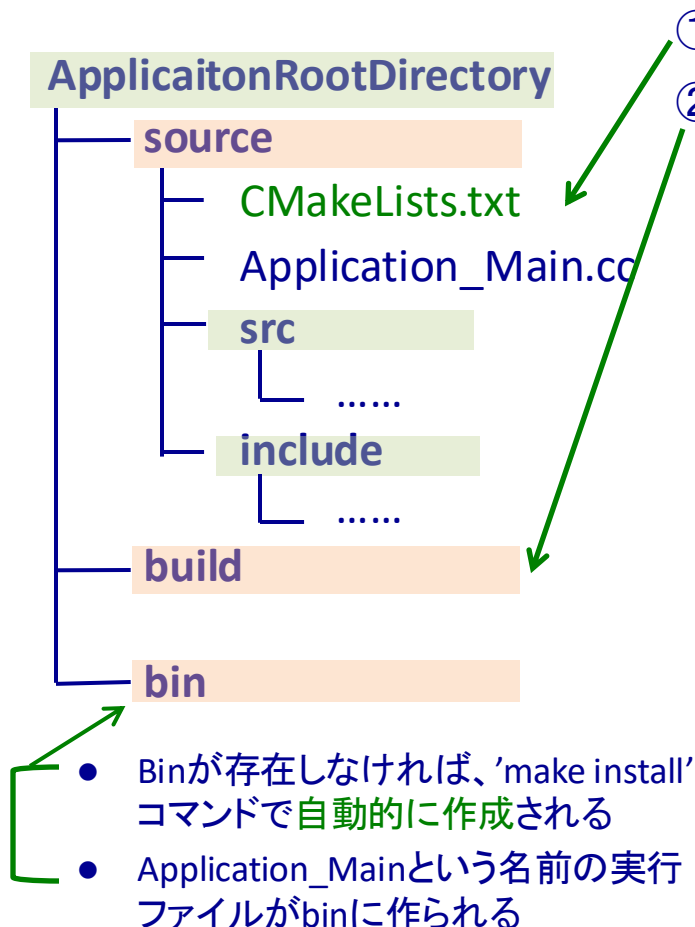
- コマンドは大小文字の区別なし(set=SET)
- 変数名、変数値は大小文字の区別あり

このコード部分は一切変更不要

→ blackboxとして扱うのでOK

提供CMakeLists.txtを使ったビルドの手順

- ApplicationRootDirectoryのもとで以下の手順に従いビルドを実行すればよい



① 前のスライドに従ってCMakeLists.txtのパラメータを設定

② 'build'という名前のディレクトリをユーザが作成して以下のコマンドを実行

```
$ mkdir build          ← buildディレクトリを作成
$ cd build
$ cmake ../source      ← MySourceにあるCMakeList.txt実行
$ make                 ← 作られたMakefileを実行
$ make install          ← アプリをbinディレクトリに保存
```

(直前のmakeコマンド実行は省略化)

[注]

- マルチコアのcpuが使えるなら '\$ make -jN' で高速化ができる
(N: 同時パラレル処理の数; 例: -j4)
- 習慣的にbuildという名前が使われるが別の名前でもOK
- 上記の順序でcmakeコマンドを一度実行しておく、あるモジュールのソースコードを変更した場合、'make install'の実行だけで、変更モジュールのみがcompile/linkされ実行ファイルに反映される
 - 全モジュールの再compile/linkが不要なので、大規模プログラム開発において不可欠な機能 (makeが持っている機能)
 - ただし、新しいモジュールを追加した場合は、cmakeの実行が必要

[注] 上に述べたビルド手順は後のHands-onで実習する

cmake使用のまとめ

cmakeを使用するための最低限知識まとめ

■ cmakeの使用手順

```
$ mkdir build
$ cd build
$ cmake path_to_CMakeLists.txt. (前のスライド例では './source')
$ make;
$ make install
```

← 'make install'だけでもOK

■ ソース ディレクトリ

- ビルドの対象となるソースファイルが置かれる
- ビルドの手続きを記述したCMakeLists.txtがディレクトリの直下に置かれる

■ ビルド ディレクトリ

- cmakeのワーク ディレクトリ
 - ・ ソース ディレクトリとは独立に作ることでお互いの干渉を排除できる
- ここに以下のようなファイルが自動的に作られるが、ユーザは中身を知る必要なし
 - ・ Makefile
 - ・ ライブラリ
 - ・ アプリ実行ファイル (make installでbinディレクトリにコピーされる)
 - ・ 各種ビルド関連で作られるファイル
- buildディレクトリで一度cmakeを実行しておく、以後、ソースコードを変更しても、'make install'を実行するだけでOK

← 新しい.cc/.hhファイルを追加した場合は'cmake'をまず実行する必要がある

Geant4公式提供のExampleコードの本講習会手法でのビルド

- 本講習会で使用している“CMakeLists.txt”
 - Geant4公式Example Codesで提供されているものと異なる
 - 理由は、Geant4公式提供のものは汎用性に欠けるため
 - ユーザが作ったアプリのビルドは本講習会提供のものを使用することを推奨
- Geant4公式提供のExampleコード(basic/B1/B2,...)を本講習会手法でビルドする方法
 - 以下に 公式Example basic/B1を例に、本講習提供のCMakeLists.txtでビルド手順を示す
 - 1) Geant4が提供する B1 dir全体をユーザの任意のdirの下にsourceという名前にして全コピー

```
$ cp -r G4ExampleDir/B1 your-work-dir/source
```
 - 2) sourceの下にあるCMakeLists.txtを本講習会用のものに置き換える

```
$ cd source
$ cp (本講習会提供の)CMakeLists.txt .
```
 - 3) 上でコピーしたCMakeLists.txtの初め部分にあるビルドプログラムの名前設定を以下のようにする

```
set( NAME_MAIN_PROGRAM exampleB1 )
```
 - 4) ‘your-work-dir’に戻り、前スライドに述べた手順に従い、build dirを作って、ビルドする

[注] 公式Geant4 Exampleが提供するCMakeList.txtの使い方は以下のweb pageを参照

http://geant4-userdoc.web.cern.ch/geant4-userdoc/Doxygen/examples_doc/html/README_HowToRun.html

講義 終了